A Study of the Impact of Target Variable Transformations on the Quality of Time Series Forecasting Models

English version of the project report

Dataset: M4 Daily

Contents

1. Research Problem and Hypothesis

The purpose of this project is to determine which transformations of time series are most effective for different structural types of daily data when using both classical statistical models and modern gradient boosting models.

The central hypothesis is that there is no universal transformation that works best for all time series and all model classes. Variance-stabilizing transformations, such as Box-Cox, are expected to be more useful for series with clear heteroskedasticity. Differencing is expected to be more effective for series with a strong linear trend. In addition, global machine learning models such as CatBoost are expected to benefit more from standardization inside transformations than local statistical models.

The study is limited to daily time series from the M4 Daily subset. The forecasting horizon is fixed at 14 days, and the historical input window for the global model is set to 30 days.

1) Only daily data are considered. The experiments use the M4 Daily subset.

2) The forecasting horizon is fixed at 14 days, while the input history window is fixed at 30 days.

3) The starting date of sampled time series is cut at 2010-01-01. The reason is that the 2008-2009 crisis period could introduce a structural break and extreme volatility. Applying transformations across such a break might produce explosive or near-zero transformed values, distorting more stable periods that are better suited for identifying trend and seasonality.

4) Because the data are daily, weekly seasonality is used. The exploratory analysis of the M4 Daily sample did not reveal a strong and unambiguous seasonal structure, but a weekly cycle is a common and reasonable assumption for daily data, especially because human activity often differs between weekdays and weekends.

5) The global model is trained within each cluster separately, following a cluster-based global modelling approach. The feature set for CatBoost is intentionally restricted to lagged target values and the series identifier. Additional features are omitted because the project is focused on the effect of target transformations, and extra covariates could confound the interpretation.

6) CatBoost was trained with 1,000 iterations and early stopping after 50 non-improving iterations in the original experiment. Since the model was trained under time constraints, some underfitting is possible. This does not contradict the research goal, because the relative effect of different transformations remains visible even under a constrained global model setup.

The analysis refers to results and plots produced in the Jupyter notebook analysis_results.ipynb. The repository structure and instructions for running the project are described in README.md.

2. Dataset Description and Preprocessing

The experiments use the M4 Dataset, Daily subset. A random sample of 200 time series is selected. The sampling uses random_state=42 wherever randomization is required. This sample size is sufficient for testing the hypothesis while keeping cross-validation and model training computationally feasible.

2.1 Preprocessing steps

1) Synchronization. All time series are converted to a common daily frequency.

2) Missing values. Linear interpolation is used, supported by backward fill and forward fill where necessary.

3) Cleaning. Duplicate index values are removed.

4) Stabilization for Box-Cox. Since Box-Cox requires strictly positive values, a minimal positive shift is applied if zeros or negative values are present.

In the sampled M4 Daily data, all values were already positive and no internal missing values were found. Therefore, the preprocessing steps related to shifting and interpolation served mostly as safeguards against possible runtime errors rather than as active transformations of the selected data.

The preprocessing log reported the following summary:

- Data loading and filtering start date: 2010-01-01.
- Minimum series length: 150 days.
- Total number of collected series: 200.
- Series with internal missing values: 0.
- Series requiring a shift because of non-positive values: 0.
- Minimum series length after filtering: 150 days.

3. Methodology: Clustering and Validation

Instead of clustering raw time series directly using shape-based distances such as Dynamic Time Warping, this project applies feature-based clustering. For each time series, statistical features are extracted using the tsfeatures library.

The following feature groups are used:

- Trend and seasonality: the strength of trend and periodic structure.
- Entropy: the complexity and unpredictability of the series.
- Nonlinearity: the extent to which the series deviates from a linear dynamic.
- Stability and lumpiness: the stability of variance and the presence of sudden local bursts or regime changes.

The selected features are directly connected to the hypothesis. The transformations studied in the project are designed to address specific time series problems: heteroskedasticity, non-stationarity, trend, and noise. Clustering by features such as lumpiness, trend, and stability groups series by their statistical structure rather than by superficial shape similarity. This makes it possible to ask which transformation is best suited to a particular structural problem.

3.1 Justification of weekly seasonality

During exploratory analysis, autocorrelation and partial autocorrelation plots were examined. The correlograms did not reveal strong seasonal peaks. However, for daily data, a weak weekly cycle is still plausible due to weekday-weekend differences in human behavior. Therefore, seasonality was fixed at 7 for statistical models such as AutoARIMA and AutoETS and for the calculation of MASE.

3.2 Clustering algorithm and interpretation

The project uses hierarchical agglomerative clustering with Ward linkage, which minimizes within-cluster variance. The silhouette score suggested that two clusters could be optimal. However, for greater experimental variability and a more informative comparison, three clusters were selected. Two of the clusters are structurally similar but still differ in features that are meaningful for distinguishing time series types.

The resulting distribution of time series across clusters is:

Cluster	Number of series
0	125
1	40
2	35

The clusters can be interpreted as follows:

- Cluster 0 - stable series with visible trends. These series have clear trends (Trend approximately 0.995) and moderate local noise (Entropy approximately 0.276). Nonlinearity is very low. They are readable, but include local fluctuations. This cluster is similar to Cluster 2, although noisier. CatBoost may benefit from learning local noisy patterns in this group.
- Cluster 1 - noisy series with outliers. These are the most difficult series to forecast. They have the highest uncertainty, the highest nonlinearity (around 0.182), and pronounced heteroskedasticity. Their variance changes over time (Lumpiness around 0.083), and the trend is weaker. Classical models may require strong transformations to handle such data.
- Cluster 2 - trending and nonlinear but highly predictable series. These series have very smooth and stable trends (Trend around 0.998, Stability around 0.999), strong persistence, minimal noise, and almost no variance jumps. They are expected to be easier for classical statistical models.

4. Validation Protocol

The experiments use walk-forward validation with three folds. This protocol is designed to evaluate models in a way that resembles a real forecasting setting and prevents leakage from the future.

The procedure cuts test windows from the end of each series and moves backward by the size of the forecasting horizon, which is 14 days.

- Fold 1: the model is trained on all historical data except the last 42 days, and the test interval is [N-42, N-28].
- Fold 2: the training set is extended by 14 days, and the test interval shifts forward to [N-28, N-14].
- Fold 3: the model is trained on all data up to the last 14 days, and the test interval is [N-14, N].

This protocol tests the stability of forecasting performance across different recent periods and imitates a production scenario where the forecasting pipeline is periodically retrained with newly available data.

4.1 Protection against data leakage

1) Transformation isolation. The TimeSeriesTransformer is fitted strictly on the training part of each fold. All parameters, including the Box-Cox lambda, mean, standard deviation, and clipping bounds, are calculated only from historical data.

2) Target isolation. The 14-day forecasting horizon is separated from the time series before model fitting. The model never observes the target values from the test interval before metric calculation.

3) Baseline isolation. For MASE, the naive model error used for scaling is computed only on the training segment of the corresponding fold. Therefore, the scale of the error does not depend on test data.

4.2 Internal validation for the global model

The global CatBoost MIMO model uses autoregressive windows as input features and is prone to overfitting. To control overfitting without using the outer test set, an additional internal split is used within each training fold. The last 10% of generated windows are separated as an internal validation set. CatBoost is trained on the remaining 90%, while the internal validation set is used for early stopping. If MultiRMSE does not improve for 50 iterations, training is stopped.

5. Experiments and Models

The study compares four target variable transformation strategies. All transformations are fitted only on the training part of each fold to avoid leakage.

1) None. The original target scale is used as the baseline transformation.

2) Log1p. The transformation log(1 + y) is applied. It is a classical variance-stabilizing transformation for series with exponential growth or heteroskedasticity.

3) First-order differencing. The model predicts increments rather than absolute values. During inverse transformation, forecasts are reconstructed using a cumulative sum added to the last known training value.

4) Box-Cox with robust inverse transformation. This is the central transformation in the experiment because it is theoretically suitable for variance stabilization.

5.1 Robust Box-Cox implementation

A standard Box-Cox implementation can be unstable in forecasting pipelines. If a model predicts an abnormally high value in the transformed space, the inverse transformation may explode, producing infinities or NaN values. This breaks metrics such as RMSE and MASE for the entire fold. To solve this issue, a custom TimeSeriesTransformer was implemented.

1) Standardization of the transformed space. After estimating the optimal lambda parameter and applying Box-Cox, the transformed training series is standardized using the training mean and standard deviation. This is crucial for the global CatBoost model, because it places series with different original scales into a comparable numerical range.

2) Dynamic robust clipping. Instead of hard-coded limits, the admissible transformed range is computed for each series. The 1st and 99th percentiles are extracted from the transformed training series, and a 50% margin is added. Predictions are clipped to this range before inverse transformation. This allows the model to forecast new highs while preventing numerical explosions.

3) Boundary protection. The inverse Box-Cox transformation has a mathematical boundary. If the model prediction crosses this boundary, the value is moved back into a safe range using a small epsilon.

4) Fallback to the naive forecast. If inverse transformation still produces NaN or infinite values, the prediction is replaced by the last known training value. This softly penalizes invalid complex-model forecasts by reducing them to a naive baseline rather than letting the whole validation fold fail.

5) Handling of constant series. Constant time series make scaling degenerate and make Box-Cox unnecessary. A detector for constant series disables transformations and returns the constant value as the forecast. In the M4 sample, this is mainly a defensive measure.

5.2 Model families

The experiment includes two modelling paradigms: local statistical models and a global machine learning model.

Local baselines are used to establish a lower bound for useful performance. The Naive model predicts the last observed value for the whole horizon. The Seasonal Naive model predicts the value observed exactly one season earlier, with a seasonal period of seven days. These baselines are essential for interpreting MASE.

Classical statistical models are implemented through StatsForecast: AutoARIMA, AutoETS, and AutoTheta. These models are trained separately for each time series. Their automatic hyperparameter selection is useful not only for forecasting, but also for interpreting the effect of transformations.

- If differencing successfully removes non-stationarity, AutoARIMA is expected to reduce or disable its own differencing parameter d.
- If Box-Cox successfully stabilizes heteroskedasticity, AutoETS is expected to shift from multiplicative components to additive components.

The machine learning approach is a global CatBoost model trained within each cluster. It learns from all series in the cluster simultaneously and attempts to capture general patterns shared across structurally similar time series.

Training uses sliding autoregressive windows. Each time series is split into overlapping windows. The 30 most recent values are used as numerical lag features. The series identifier ts_id is added as a

categorical feature. CatBoost processes it through its internal categorical feature mechanism, allowing the model to preserve part of each series identity while still learning global patterns within a cluster.

The global model follows the MIMO strategy: for each input window, it predicts the full 14-day future vector in one pass. The loss function is MultiRMSE, which optimizes the distance between the predicted 14-dimensional trajectory and the actual trajectory. This encourages the model to predict the future path jointly rather than point by point.

5.3 Evaluation metrics

Three metrics are used because they highlight different aspects of forecasting quality:

1) sMAPE. This metric allows comparison across time series of different scales. A modified version is used to remain stable for small denominators.

2) MASE. This is the key metric in the study. It measures whether a model is better than a naive seasonal forecast. If MASE is greater than 1, the model is worse than the baseline.

3) RMSE. This metric penalizes large errors and outliers. It is especially important for evaluating the stability of transformations such as Box-Cox, whose inverse may produce large errors.

The aim is not to find an absolutely optimal forecasting model for the M4 Daily subset, but to compare transformations under the same experimental conditions. Differences in metrics across models and transformations provide evidence about which transformation is suitable for a particular modelling paradigm and data structure.

6. Training Procedure and Technical Details

The full experiment is organized as a reproducible computational pipeline.

1) Data loading and clustering. The pipeline loads 200 sampled M4 Daily series, extracts time series features using weekly seasonality, scales the feature matrix, and assigns series to clusters. A dictionary is formed where each key is a cluster number and each value is the list of series belonging to that cluster.

2) Validator initialization. A WalkForwardValidator is created with the global experiment parameters: horizon = 14, number of folds = 3, seasonality = 7, and history window = 30.

3) Main experimental loop. The pipeline iterates over clusters. Inside each cluster, it iterates over transformations: none, log1p, Box-Cox, and first-order differencing.

4) Training the global model. For each cluster-transformation pair, CatBoost cross-validation is executed first. The model is trained on all series from the current cluster. After training, the actual number of trees selected by early stopping and the best MultiRMSE loss are logged. The trained model is serialized to disk in .joblib format.

5) Training local statistical models. For the same cluster-transformation pair, local baselines and statistical models are trained through StatsForecast. Unlike CatBoost, these models are fitted separately for each series, but their forecasts and metrics are aggregated.

6) Metric aggregation and storage. Metrics from all folds, models, and transformations are collected into a Pandas table and saved as metrics.csv. The file is used later for plots and analytical conclusions in the Jupyter notebook.

7. Results and Discussion

The figures used in the discussion are available in the Jupyter notebook.

7.1 Impact of transformations on time series structure

The classical statistical models serve not only as forecasting tools, but also as diagnostic instruments for the structure of the data.

- Identification of heteroskedasticity. On the original scale, AutoETS selected multiplicative models in 272 cases: ETS(M,N,N) in 206 cases and ETS(M,A,N) in 66 cases. This confirms that many M4 Daily series are heteroskedastic: their variance grows with the level of the series.
- Additive effect of transformations. After applying transformations, multiplicative AutoETS components almost disappear. Additive models dominate, for example ETS(A,N,N) appears in 379 cases under log1p. This indicates that the transformations linearized the series and stabilized variance.
- AutoARIMA reaction to differencing. On raw data, AutoARIMA often uses seasonal integration and moving average terms. When external differencing is applied, AutoARIMA frequently switches to models such as ARIMA(0,0,0)(0,1,0) or ARIMA(0,1,0)(0,1,0). Because the series have already been made more stationary by differencing, the internal mechanisms of AutoARIMA are reduced or changed to avoid over-differencing.

8. Analysis of Differencing and the Extrapolation Problem in Tree Models

The CatBoost MIMO results reveal a core limitation of gradient-boosted decision trees: weak extrapolation ability.

- Failure on none and log1p. On raw values and logarithms, CatBoost performs poorly. MASE ranges from 4.40 to 9.18 on the original scale and from 1.27 to 2.23 with log1p. RMSE reaches extremely large values, up to approximately 1525. The reason is that decision trees do not extrapolate well outside the range of the training window. If a trend continues upward, CatBoost tends to saturate near historical levels and produces a flat forecast.
- Differencing as a solution. First-order differencing changes the problem from forecasting absolute values to forecasting increments. In that transformed space, CatBoost becomes much more effective. Its MASE drops to 0.96 in Cluster 0, 0.92 in Cluster 1, and 1.11 in Cluster 2.
- Conflict between differencing and statistical models. Differencing improves CatBoost but harms AutoARIMA and AutoETS. Their MASE increases to approximately 2.3-2.8 under differencing. These models are designed to handle trends internally. External differencing followed by cumulative reconstruction accumulates forecast errors across the horizon.

9. Model Performance by Cluster

The MASE results show that the best transformation depends both on the structural type of time series and on the model class.

1) Cluster 2 - smooth trending series. Statistical models, especially AutoARIMA and AutoETS on none or log1p, perform best with MASE around 1.10-1.11. On very smooth trends, simple local models work well. CatBoost with differencing is close but slightly worse, because it has to learn one global logic across multiple series rather than a perfectly local trend.

2) Cluster 0 - stable trends with noise. AutoARIMA on the original scale and CatBoost with differencing both perform well, with MASE approximately 0.95-0.96. CatBoost becomes competitive because it can use information from multiple noisy series in the same cluster.

3) Cluster 1 - noisy and volatile series. CatBoost MIMO with differencing is the strongest configuration, reaching MASE around 0.92. Local statistical models struggle more in this cluster, while the global model benefits from learning shared nonlinear behavior across multiple difficult series.

10. Analysis of CatBoost Training Logs

The CatBoost training logs provide additional insight into the transformed feature and target spaces.

- None: loss from approximately 1289 to 3135 and 150-340 trees. The high loss is expected because MultiRMSE is computed on the raw scale. Training stops relatively quickly because the model reaches the limit of what tree ensembles can do without extrapolation.
- Log1p: loss from approximately 0.16 to 0.43 and 194-705 trees. The loss is small because the scale is compressed. In Cluster 0, the model builds 705 trees, suggesting that log1p creates a smooth optimization landscape where CatBoost can gradually learn smaller patterns without overfitting immediately.
- Diff: loss from approximately 390 to 1296 and 36-200 trees. Training stops very early, especially in Cluster 1, where only 36 trees are used. The differenced target contains less long-range trend signal and more stationary increments. CatBoost quickly captures the main increment dynamics, while further trees risk overfitting noise.
- Box-Cox: loss from approximately 0.7 to 8.9 and 58-273 trees. The early stopping behavior, especially 58 trees in Cluster 1, indicates instability of the transformed space and rapidly worsening validation loss.

11. Distribution of MASE

The boxplots of MASE support the numerical conclusions.

1) The MASE = 1.0 line separates useful configurations from configurations that do not beat the naive baseline.

2) For none and log1p, the boxplots for AutoARIMA, AutoETS, and AutoTheta are compact and close to 1.0 or slightly below it. This indicates stable performance of local statistical models. CatBoost under these transformations is far above the visible upper range of the plot.

3) For differencing, the CatBoost distribution moves down toward the MASE = 1.0 line and becomes more stable, showing that differencing makes the global tree model much more competitive.

4) For Box-Cox, the distributions are stretched and their medians shift upward. This visually confirms that Box-Cox introduces unstable forecasting behavior in this experiment.

12. Why Box-Cox Failed in This Experiment

One part of the initial hypothesis was that Box-Cox would perform best on heteroskedastic series. This part of the hypothesis is rejected by the experiment.

Even with quantile-based clipping and robust inverse transformation, Box-Cox inversion on noisy daily M4 data inflates forecast errors. MASE increases to values between approximately 2.65 and 9.18. In contrast, log1p is safer and more robust as a variance-stabilizing transformation in automated forecasting pipelines.

13. Final Conclusion on the Hypothesis

The project achieved its goal: it identified how different target transformations affect forecasting quality for different model classes and time series structures.

1) There is no universal transformation. This part of the hypothesis is confirmed. Differencing harms local statistical models but strongly improves the global CatBoost model.

2) Box-Cox is not the best transformation for heteroskedasticity in this setup. This part of the hypothesis is rejected. Box-Cox is too unstable during inverse transformation. Log1p is a safer method for variance stabilization.

3) Differencing is more effective for linear trends only in the case of gradient boosting. This part of the hypothesis is partially confirmed. Differencing solves the extrapolation limitation of tree models, but classical models can handle trends internally and are harmed by external differencing.

4) Global models benefit more from transformation-based standardization than local models. This part of the hypothesis is confirmed. Differencing turns CatBoost into the best model on the most difficult cluster, with MASE around 0.92, while external transformations do not radically improve local statistical models.

In the post-crisis period considered in this study, AutoARIMA with log1p is appropriate for stable trending series, while CatBoost MIMO with first-order differencing is the best choice for complex, volatile, and noisy series.

A key limitation is that the selected series do not show strong seasonality. Because MASE is scaled against a seasonal naive forecast, it may not be the perfect metric for all series in this sample. However, this limitation is mitigated by comparing several metrics, several baselines, and multiple model classes. The discrepancy between SeasonalNaive and the seasonal naive component embedded in MASE is likely connected to library-specific implementation details.

Although sMAPE and RMSE are not the main focus of the final conclusion, they were used in the analysis to assess transformation robustness, model stability, and the reliability of inverse transformations.

Appendix A. CatBoost Training Logs

Cluster	Transformation	Trees	Loss
Cluster 0	none	340	1289.7378
Cluster 0	log1p	705	0.1664
Cluster 0	boxcox	114	2.3316
Cluster 0	diff	200	441.4919
Cluster 1	none	195	3135.6997
Cluster 1	log1p	347	0.4330
Cluster 1	boxcox	58	8.9930
Cluster 1	diff	36	1296.0384
Cluster 2	none	157	1299.3789
Cluster 2	log1p	194	0.1653
Cluster 2	boxcox	273	0.7128
Cluster 2	diff	44	390.8604